

PRISM: Evidence-Admission Contracts for LLM-Assisted Repository Vulnerability Analysis

Sri Sushmitha Janapareddy

Vishal Chandupatla

Internal Review Draft: Not for External Distribution

Abstract

Large language models increasingly assist repository vulnerability analysis, but an authorized, correctly attributed assessment can still be false. We investigate when such assessments may authorize terminal security decisions through PRISM, an implemented system combining persistent obligations, bounded semantic questions, explicit premise origins, identity-bound evidence, and class-specific discharge policies. A primary-artifact audit reconstructs its current admission contract and two retained authorization controls. Completed, prespecified Phase-2A falsification controls expose three assurance limits. A false but policy-permitted semantic premise enabled a terminal safe decision while its matched truthful assessment remained unresolved. An independently infeasible attacker-to-sink path received a consumed static vulnerable decision through the production extraction and structural-validation route, as did its minimally changed feasible control. Capability inspection found no supported semantic-fact replacement or retraction route with dependent consumed-proof revocation. These findings show that provenance and authority do not imply semantic truth, current candidate-local admission does not guarantee feasible-path composition, and the tested revocation capability is absent. They do not estimate population error rates or establish overall ineffectiveness. The completed outcomes are supplied experimental reports whose checkpoint identities match the audited artifacts; independent verification of their linked raw execution records remains outstanding. The remaining research hypothesis is whether the narrower origin/discharge contract provides incremental value over an equally durable, equally equipped baseline. We specify that matched comparison, but do not claim comparative benefit or established novelty.

1 Introduction

When may an LLM-generated security assessment become a terminal security decision? This question arises when an LLM-assisted analyzer moves from exploring a repository to reporting a vulnerability, accepting a defense, or closing an investigation. A model may return valid structured output, cite authentic source text, and answer an authorized question while misunderstanding an authorization guard or sanitizer. Several individually plausible observations may also describe incompatible execution paths, and evidence may become stale when dependent facts change. Attribution is therefore distinct from semantic truth: correctly cited and correctly authorized does not necessarily mean correct. A durable work queue preserves tasks, but does not by itself define when the evidence justifies discharging an obligation.

Existing systems already combine model reasoning with systematic program analysis. IRIS infers specifications for static analysis; LLift examines difficult static warnings; NESA composes neural and symbolic relations; and RepoAudit organizes repository exploration and local reasoning [7, 11, 12, 19]. Contemporary agentic pipelines also separate candidate selection from bounded workers [5]. Improving the surrounding contract consequently requires a more precise claim than introducing enumeration, local reasoning, or a worklist.

We study **evidence admission**: the rules determining whether an assertion may enter a decision and whether the resulting derivation may discharge a persistent obligation. This contract should identify the analyzed artifact, the question

asked, the producer’s permitted authority, the evidence cited, the policy applied, and the transition that consumes the decision. It should also expose what remains unresolved. Such records support attribution and inspection, but their value depends on what the admission checks actually establish. A provenance record cannot substitute for a semantic checker.

PRISM is the implemented case study. Its canonical path binds obligations and evidence to an analysis identity, issues bounded model questions, checks premise-origin permissions, applies same-premise authoritative precedence, evaluates class-specific policies, and requires a consumed policy derivation before terminal transition. The audit supports these local checks. Completed Phase-2A controls show that a false permitted semantic premise can enable terminal safety and that an infeasible path can reach a consumed vulnerable decision through the production analysis route. Capability inspection also establishes the absence of the tested semantic replacement and dependent-proof revocation route. These are observed limits of the tested checkpoint, not repairs or estimates of population error.

The main research question is whether these explicit rules improve the validity and auditability of terminal decisions relative to an equally durable pipeline using the same semantic evidence and validators. The final matched comparison has not been completed. This paper reports an implementation audit, two retained Class-11 control observations, and completed Phase-2A falsification results. The latter narrow the defensible assurance claim while leaving the incremental-benefit hypothesis open. Section 5 separates verified checkpoint in-

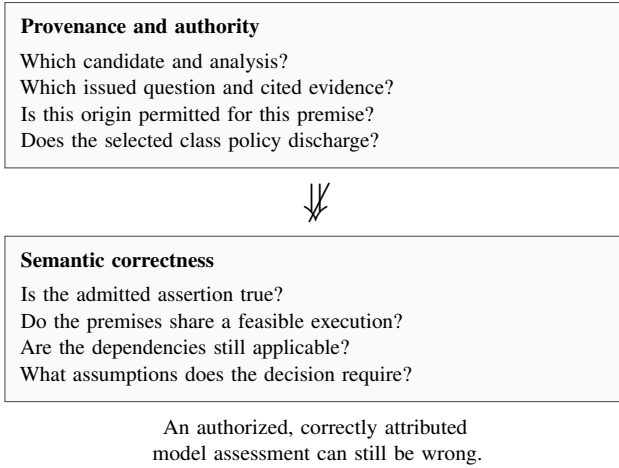


Figure 1: Admission can verify attribution, permission and evidence identity while a semantic assertion remains false. The missing implication is central to the study, not a measured error rate.

tegrity from the supplied reports’ execution evidence and its verification limits.

The contributions are: (1) a characterization of evidence admission that separates provenance and permission from semantic correctness; (2) an account of PRISM’s implemented candidate-local admission and discharge contract; (3) a primary-artifact audit, including two current Class-11 controls with necessary model semantics and independently supplied fixture-specific closure; (4) empirical falsification of three assurance interpretations of the current implementation, through two executed counterexamples and one capability-inspection result; and (5) a matched evaluation design that holds semantic inputs and validators fixed. Specifically, provenance/authority does not imply semantic truth, current generic admission does not guarantee feasible-path composition, and dependency-driven semantic proof revocation is not implemented. These findings do not establish that the surrounding architectural ingredients are new or that the proposed comparison has succeeded.

2 Related work

2.1 Hybrid repository and dataflow analysis

IRIS combines model-assisted source/sink specification inference with CodeQL-based analysis, while LLift applies local model reasoning to initialization warnings from an upstream analyzer [11, 12]. LLMDFA constructs and composes dataflow information using models and supporting analysis procedures [17]. These systems establish substantial antecedents for allocating bounded semantic tasks around systematic analysis. PRISM’s use of a finite candidate population cannot itself establish a new paradigm.

NESA is particularly close to the broad architectural thesis: symbolic, neural, and intensional relations support relational evaluation with local neural facts and fixed-point computation [19]. Its relation types and evaluation machinery already separate neural interpretation from external composition. PRISM’s research question concerns the admission

of particular premise origins into terminal policies, not the invention of neuro-symbolic composition or fixed points.

RepoAudit organizes repository-level investigation using local analysis, memory, and validation. LLMSAN checks model-generated reports with dataflow-oriented consistency procedures [7, 18]. Such validation must not be characterized as absent merely because it differs from PRISM’s policy representation. Nor should model-assisted path checks be described as general formal feasibility proofs. OpenAnt, an open-source system described in a preprint, further combines code decomposition, adversarial verification, and dynamic testing [8]. Its publication status and verification approach are distinct from a peer-reviewed soundness result.

Code Property Graphs and CodeQL provide earlier foundations for representing and querying program structure and dataflow [6, 21]. PRISM’s typed repository representation inherits the general benefits and limitations of program abstractions. A graph’s existence does not establish that every relevant edge or feasible execution has been represented.

2.2 Checked evidence, partial verification, and revision

Proof-Carrying Code separates an untrusted producer from a checker validating a formal safety argument [14]. LEMUR uses model-proposed intermediate properties with external verification and recursive discharge, conditional on the assumed verifier’s soundness [20]. These provide stronger checking semantics than accepting an authorized semantic assertion because its schema and citations are valid. PRISM’s policy derivations must be interpreted at their actual assurance level.

Agentic Interpretation, currently a preprint, explicitly discusses evidence-backed assessments, stabilization, confidence bases, and revision of evidence [13]. Its implementation and evaluation are presented as future work, but that does not erase its conceptual overlap. Conditional Model Checking communicates conditions describing verified and residual state spaces; CoVeriTest coordinates testing around covered and open goals [2, 3]. Residual uncertainty and continued work therefore have substantial pre-LLM antecedents. PRISM’s obligation counts are not equivalent to a formally characterized residual program-state space.

2.3 Contemporary security systems and evaluation

Cognition’s Agentic MapReduce describes deterministic selection, bounded workers, persistent candidate work, and reduction; it also recognizes that selectors can impose a recall ceiling [5]. Codex Security documentation describes partial or unknown coverage and cautions against interpreting a missing finding as proof of a fix [15]. These are vendor technical sources, not independent evidence that PRISM performs better or that competitors lack explicit completion semantics.

Claude Code Security, Capital One VulnHunter, and Visa’s Vulnerability Agentic Harness likewise describe contemporary workflows involving model reasoning, validation, and orchestration [1, 4, 16]. VulnHunter and VVAH expose public code; that is a separate attribute from peer review or independent performance reproduction. VVAH documents host-derived status and staged validation, while its reserved graph structures do

not establish general path-sensitive analysis. We do not infer either a competitor’s missing internal contract or PRISM’s superiority from incomplete public documentation.

SEC-bench and the SEC-bench Pro preprint motivate reproducible security-task evaluation with explicit task and grading boundaries [9, 10]. Reproducing a disclosed vulnerability, discovering an unknown one, and deciding whether a generated finding is justified are different estimands. The proposed PRISM study measures the incremental effect of admission on a fixed candidate population; it cannot by itself measure repository-wide discovery recall.

Together, this literature substantially narrows the proposed contribution. The plausible distinction is the exact implemented admission contract and its observable consequences under controlled comparison. No uniqueness claim follows from the absence of PRISM’s terminology in another paper.

3 System model and admission semantics

Let R denote a repository artifact and I an immutable analysis identity binding the relevant source, scope, analyzer, registry, and policy configuration. Let O be the set of persistent obligations generated under that identity. An obligation $o \in O$ belongs to a class c and a declared universe/member context. A security premise p is an assertion such as attacker influence, an authorization predicate, or complete relevant paths. Let $A(p)$ be its permitted evidence origins under the applicable class policy, and $E(p)$ the evidence admitted for it.

An evidence assessment contains a premise identifier, status, origin, producer, and cited evidence references. Its status is satisfied, refuted, unknown, or conflict. The notation abstracts the inspected implementation rather than specifying an independently verified semantics for the source language. In particular, a citation identifies evidence; it does not establish that the cited text entails the premise.

Let $E = \bigcup_p E(p)$ collect the evidence admitted for the obligation, and let $E_\pi \subseteq E$ denote the effective support selected for a proposed terminal derivation π . Let P_c be the class-specific terminal policy, including its required premise combinations, alternatives, and disqualifiers, and σ_E the aggregate statuses after precedence. Let $D_t(o)$ denote the decision recorded by the admitting transition at time t . The canonical admission requirement can be summarized as

$$\begin{aligned} \text{Admit}_t(o, \pi, I) = & \\ & \text{Bind}_t(o, E, \pi, I) \\ & \wedge \text{OriginsAllowed}(E, P_c) \\ & \wedge \text{PolicyAccepts}(P_c, \sigma_E) \\ & \wedge \text{NoBlockingConflict}(P_c, \sigma_E) \\ & \wedge \text{NonModelSupport}(E_\pi). \end{aligned} \quad (1)$$

Here, binding includes the identity and issued-evidence checks applicable to the route. Non-model support captures the inspected engine’s requirement for at least one non-model origin among selected terminal-supporting premises. These predicates summarize checks at admission/consumption; they do not assert continuing validity after dependency changes. A

newly recorded terminal decision additionally requires authorized consumption of the admitted derivation:

$$\begin{aligned} D_t(o) \in \{\text{vulnerable}, \text{defended}\} \\ \implies \text{Admit}_t(o, \pi, I) \wedge \text{Consumed}_t(o, \pi). \end{aligned} \quad (2)$$

This is a summary of the local transition contract, not a theorem about program behavior. In particular,

$$\text{Admit}_t(o, \pi, I) \not\Rightarrow R \models \text{Property}(o). \quad (3)$$

The missing implication matters when a permitted premise is false, selected facts are path-incompatible, or a dependency becomes stale. The current binding predicate does not silently include a universal path-compatibility solver or dependency-freshness theorem. Historical identity matching also differs from verifying that a decision remains current after every possible environmental change.

We use four assurance terms. A **policy-admissible decision** satisfies the admission and discharge rules. A **model-supported semantic finding** depends on at least one fallible model assertion. **Runtime-confirmed scoped behavior** is witnessed under a specified executor/environment contract. A **formally checked property** is validated by a sound formal checker under declared semantics. General PRISM findings are not assigned the fourth category merely because their internal record is called a `FindingProof`.

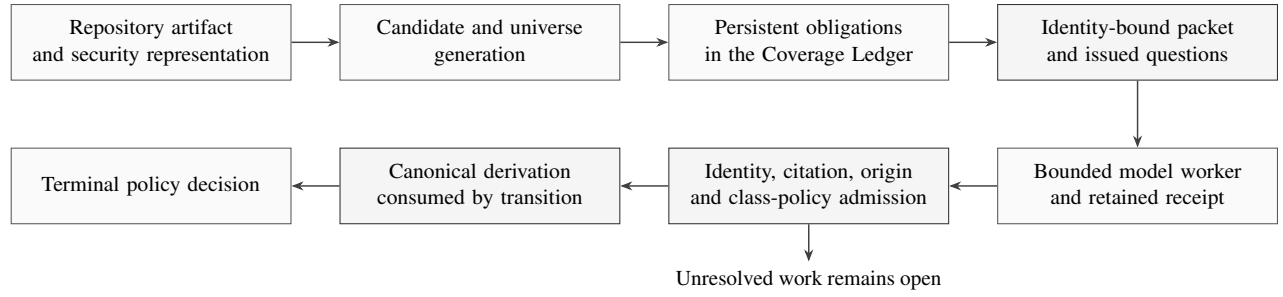
4 PRISM architecture and authority model

4.1 Evidence flow

PRISM constructs a repository/security representation containing source identities, extracted facts, and typed relationships. The canonical compiler uses system extraction and candidate generation to create class-specific universes and obligations. These universes delimit what is being analyzed; they do not enumerate all possible vulnerabilities in arbitrary programs. Repository, discovery, candidate-generation, resolution, runtime, and fixed-point completion are distinct accounting dimensions.

The Coverage Ledger persists obligations, attempts, leases, evidence, and state transitions. Candidate identities bind the analysis and universe/member context to the class, generator, and evidence. Model workers receive bounded packets with an issued question set and citation references. A response must match its candidate, packet, policy, and semantic contract before supplying model evidence. Missing evidence may request an issued expansion handle; the system controls the resulting packet and lineage.

The admission engine aggregates allowed assessments and evaluates the class policy. A terminal ledger transition consumes the resulting canonical derivation. Keeping verdict and basis separate is important: an internal STATIC basis can include model semantics and does not mean model-free or formally sound. Runtime evidence enters through a distinct request/admission boundary. The fixed-point controller can invalidate completion and schedule review when admitted memory changes, with the freshness limitations discussed below.



Scoped runtime evidence has a separate admission route. Review adds obligations and preserves generation history.

Figure 2: PRISM evidence flow. Persistent obligations connect bounded model assessments and system evidence to admission and consumed terminal derivations; unresolved work remains attributable.

4.2 Permitted origins and precedence

The stored static-producer origins are `PARSER`, `STATIC_ANALYZER`, and `DETERMINISTIC_RULE`. Other origins are `MODEL_SEMANTIC`, `RUNTIME`, `ENVIRONMENT`, and `HUMAN_REVIEW`. `SYSTEM_STATIC` is conceptual shorthand, not an origin enum. The current catalog contains 37 premise types: 27 model-assessable and ten model-forbidden. Source/policy records A03 and A05 establish these counts and rules; they are not accuracy measurements. Appendix B preserves the complete premise catalog and the distinction between ordinary, runtime-only, and environment-only origins.

Representative model-forbidden premises are `REPOSITORY_SCOPE_COMPLETE`, `COMPLETE_RELEVANT_PATHS`, `CONTROL_SCOPE_COMPLETE`, and `OPERATION_UNREACHABLE`. The remaining forbidden categories cover advisory non-applicability and runtime/environment authority. Runtime-only and environment-only premises require their corresponding origins. Ordinary premises remain subject to the selected class’s allowed-source policy; model-forbidden closure is not universally restricted to exactly the three static-origin labels.

The model can see authoritative premise names in packet policy data. The boundary is that these premises are not issued as authoritative model questions and model assertions cannot directly establish them. After validation, the canonical worker fixes the origin of its evidence to `MODEL_SEMANTIC`; a response cannot choose a stronger origin merely by including that label.

For the same premise, non-unknown authoritative evidence takes precedence over model evidence. Conflicting authoritative assertions and incompatible terminal outcomes prevent resolution. This protects against a particular conflict pattern, not every cross-premise inconsistency. A false model assertion without authoritative contradiction can still supply an allowed premise. Permission and precedence therefore define an admission discipline rather than a semantic truth oracle.

4.3 Memory and transitive authority

The audit traced the canonical model-worker-to-memory-to-projector route. Accepted proposals retain an untrusted model-proposal namespace and producer/candidate/packet linkage. Initial packets exclude evolving analysis-wide security memory. The sole shipped projection inspected, a Class-5 XML

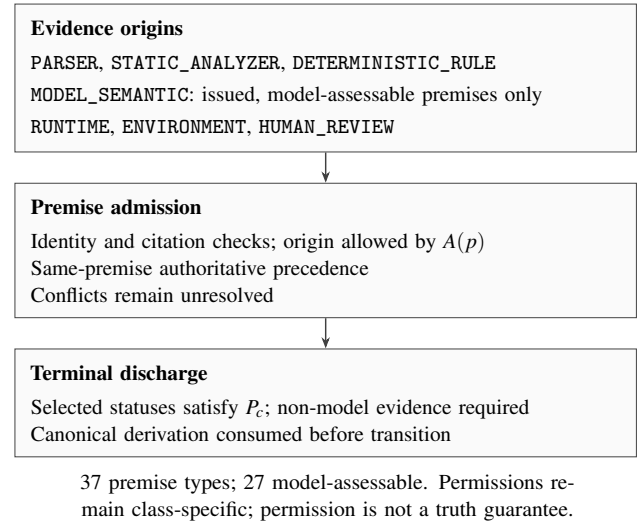


Figure 3: Premise authority and terminal discharge in the canonical path. Origin permissions and policy checks constrain admission; they do not independently prove every semantic assertion.

callable-alias projector, rereads source bytes and independently checks a narrowly specified AST/SIR shape before producing stronger evidence. Other proposals create review work rather than arbitrary authoritative facts.

No laundering bypass was found in that inspected route. This scoped finding does not establish a universal recursive authority lattice. Generic derivation records and future validator/projector extensions are not covered by a theorem that authority can never increase without independent checking. Trusted validators remain inside the system’s trust boundary. The representative chain supports a local implementation claim; it cannot justify an unrestricted guarantee over arbitrary extensions. The supporting audit groups are A05 and A08.

5 Audit method and evidence provenance

The study combines a primary-artifact audit of one source checkpoint with completed Phase-2A experiment reports bound to that checkpoint. The evidence hierarchy gives source, policy, identity, oracle, and transition records precedence over historical narratives. We distinguish inspected implementation, retained controlled observations, reported completed experiments, historical diagnostics, and hypotheses.

5.1 Retained artifact audit

Verification included archive inventory and byte hashes, manifest and executable-package consistency, declarative registry/policy reconstruction, source and test-contract inspection, and read-only SQLite reconciliation. Current control reconstruction checked candidate identities, packet/control bindings, accepted response digests, initial and final proof records, consumed-proof pointers, and attempt lineage. The original Class-11 controls’ counterfactual necessity is derived from their recorded policy; that earlier audit performed no altered-input experiment. The manuscript revision likewise ran no PRISM experiment, provider request, runtime validator, or repair.

“Current” denotes compatibility with the audited checkpoint. The two retained Class-11 controls share analyzer, registry, and policy identities but have distinct subject, analysis, and model-policy identities. They are separate cases. Historical failure, malformed-output, expansion, and runtime records form separate cohorts and are not pooled into estimates. Audit groups A00/A01 cover integrity; A03/A05 admission rules; A06/A07 retained controls; A08/A09 transitive and path analysis; A10–A12 worker diagnostics; A14/A15 runtime evidence; and A16 invalidation. These are artifact identifiers, not publications or Phase-2A experiment labels.

5.2 Completed Phase-2A controls and verification scope

The supplied PHASE2A-SUMMARY.md and PHASE2A-PAPER-FACTS.md record four synthetic scan arms and one capability check under a prespecified design. A1 uses two deterministic local injected semantic deliveries. A2 uses the real compiler, extractor, production deterministic structural validator, and admission route. A3 stops at the unsupported-capability gate. Independent fixture oracles define the A1 semantic falsehood and A2 path feasibility; PRISM’s own acceptance is not the truth criterion.

The reports record sealing the design, fixtures, oracles, and decision harness before execution, with no outcome-adjusting fixture changes, PRISM fixes, or outcome-tuning reruns. They report byte-identical pre/post inventories for the original 279-file checkpoint, including 276 source files, and its 276-source-file execution copy. They record zero live provider calls and zero Crucible/runtime-validator calls. Oracle body execution and Boolean enumeration are separate from those runtime validator calls. A3’s initial read-only metadata helper could not attribute a dataclass-generated method; a separate read-only helper completed the inventory while retaining that failure and leaving the sealed harness and PRISM unchanged.

The reports agree on result classifications and are compatible with the retained checkpoint. Read-only rehashing verified all 338 original bundle checksum entries, all 276 source-file byte hashes, the five shared checkpoint/source/analyzer/registry/policy identities, A1’s retained fixture hash, and the Class-1 and Class-11 policy hashes. Original ZIP executable bits match the manifest; ten support scripts lost those bits on local extraction without source-byte changes. Analyzer reconstruction uses recorded runtime metadata, not a recreated runtime.

A material verification boundary remains: the two supplied reports link to result JSON, the execution seal, pre/post inventories, oracle traces, proof objects, ledgers, and an interface inventory that were not supplied with them. Their result preimages, sealing chronology, execution-copy integrity, and reported proof replay were not independently reverified for this revision. The completed findings below are incorporated as reported investigator evidence; shared checkpoint compatibility is independently checked. Appendix C records exact result and input identities. The typesetting package does not contain the underlying experiment bundle. These controls are not the full matched Phase-2 comparative study.

6 Audit results and completed Phase-2A controls

6.1 Checkpoint identity and contract inventory

The supplied archive contains 340 files. All 339 indexed sizes/hashes and all 338 checksum-manifest entries reconciled. The source checkpoint contains 276 allowlisted source files; its 90-file executable package projection matches the wheel’s executable bytes. These are integrity results about supplied inputs, not evidence that their extracted semantic facts are correct. The analyzer digest was reconstructed using recorded runtime metadata; that runtime was not recreated. Complete checkpoint identifiers are retained in Table 1.

The inspected registry contains 86 unique classes and 86 explicit policy assignments across 16 families. Its five scope qualifications distinguish enumerable, bounded, heuristic, runtime-dependent, and environment-dependent work. The 37-premise catalog and 27 model-assessable entries are verified implementation counts. They do not establish 86 complete or effective detectors, comprehensive coverage, or a population-level accuracy estimate.

6.2 Retained Class-11 controlled observations

Both controls were recorded on 28 August 2026 under the audited analyzer, registry, and authorization policy. Each uses an additional CanonicalClass11ControlValidator, bound to an exact five-line fixture and prescribed AST/SIR checks. This validator supplies fixture-specific premises; it is not a general authorization or path-closure solver. The controls demonstrate how admitted origins compose under the selected policy (Table 2).

For the positive control, the vulnerable policy requires applicability, authenticated principal, attacker-controlled resource selection, sensitive action, reachable flow, complete relevant paths, and absence of the required authorization predicate. The fixture validator supplies applicability, reachability, and complete relevant paths as DETERMINISTIC_RULE evidence. One accepted response to a bounded evidence packet supplies the other four necessary premises as MODEL_SEMANTIC. The consumed final derivation has the internal verdict/basis PROVEN_VULNERABLE / STATIC; the focal candidate transitions to RESOLVED_VULNERABLE.

Removing model evidence from the recorded premise set leaves four required assertions unknown and prevents termi-

Table 1: Verified current checkpoint and control identities. Full SHA-256 values; source/package consistency does not establish semantic soundness.

Identity	SHA-256
Checkpoint manifest	4ccc4786e9ee3b0b7afe8a33fbf728b191e4c5a1ca46e08a936cb8a4f426ed2d
Source tree	35c2f2fa38faae79e2f907591b3ca6768ee41acc985358fecb16e7ab9f86aaf0
Executable package	48a01a274bbeefb57dd8bd49d7c4317edb32c3ea628a7c7230b7d8bf860bdbc7
Analyzer	c4728d157a12c737fa6d282df7028a9b23ff5a0e519be040f62d88115843f2db
Registry	68b0a474e24a7f06b813cf77b510157c56dd35fc73c15a7d6fbbae4f38e3f647
Policy root	9f5ae694372840842843d68c404a6d7ea969ae5311c9a0dd1b247aa11a39611d
Class-11 policy	d43b93ee7edfbbe3da26dfe0dea76e617c6e0620c37f84ad9d37931355625f9d
Positive analysis	0f62fc9c11deda4c6a98b2497774997a10fea6669798788aca44fed3e906378b
Negative analysis	c2e545de647e9246a94e4efac513dd662c9aa1da6a8890a391fdf103801921bb

Table 2: Current Class-11 controls under the audited policy. Initial state is the proof verdict; both focal candidates begin PLANNED. System evidence is fixture-specific DETERMINISTIC_RULE evidence.

Control	Initial state	System contribution	Model contribution	Final policy verdict	What it demonstrates	Limitation
Positive	Unresolved	Applicability; reachable flow; complete relevant paths	Authenticated principal; attacker selector; sensitive action; authorization absent	PROVEN_VULNERABLE / STATIC	Model semantics and closure both necessary	Exact local fixture; no general semantic/path proof
Negative	Unresolved	Closure; authorization dominates; absence refuted	Applicability, deliberately withheld by control validator	PROVEN_SAFE / STATIC	Model applicability plus mitigation/closure	Model agrees with mitigation; no live contradiction test

nal vulnerability. Removing COMPLETE_RELEVANT_PATHS also prevents terminal vulnerability, although the remaining assertions can support a nonterminal supported-vulnerable state. These are policy counterfactuals over recorded evidence, not new replay results. They establish that both model semantics and non-model closure are necessary to this selected derivation, not that a model is algorithmically necessary for authorization analysis.

The negative control satisfies the safe alternative of applicability, complete relevant paths, and authorization dominance. Deterministic evidence supplies mitigation and closure and refutes authorization absence. The model supplies applicability, yielding PROVEN_SAFE / STATIC and a RESOLVED_DEFENDED candidate state. The control deliberately withholds deterministic applicability to prevent pre-provider completion. The model agrees with mitigation; this is not a live conflicting-model test. Removing its applicability assertion or the required closure blocks the selected safe derivation.

Both fixtures refer to an undefined account collection and omit complete authentication, data, and deployment semantics. Their closure and reachability assertions depend on the prescribed fixture interpretation. Neither control demonstrates an executable protected-account application, a runtime exploit, or a general all-path property. Both final proof records have empty semantic/environment-assumption lists despite these dependencies; empty lists do not establish assumption-free conclusions.

Each current ledger retains seven candidate identities: one terminal focal candidate, three planned candidates, and three

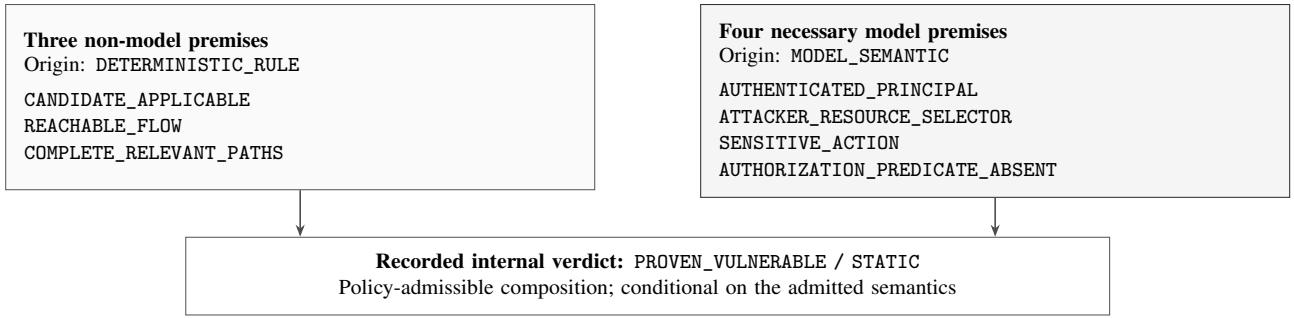
unresolved review obligations. Candidate IDs and event references reconciled at set level. Later review generations remain unfinished while the original focal proof remains consumed. Thus the observed terminal decision does not imply complete repository or fixed-point resolution. No population accuracy, cost advantage, or comparative benefit is inferred from these cases.

6.3 A1: False permitted semantic premise

A1 records A1_POLICY_ADMISSION_ACCEPTS_FALSE_SEMANTIC. Both deliveries use the same source, candidate, evidence packet, and fixed non-model premises. Only the status of AUTHORIZATION_PREDICATE_DOMINATES changes: the truthful arm returns REFUTED; the false arm returns SATISFIED. Other issued semantic premises remain UNKNOWN. Deterministic fixture evidence supplies applicability, reachability, and complete relevant paths, with no authoritative same-premise dominance contradiction.

Both responses are admitted. The truthful arm remains UNRESOLVED / NEEDS_EVIDENCE. The false arm produces a consumed PROVEN_SAFE / STATIC derivation and reaches RESOLVED_DEFENDED. Its assumptions list is empty. The independent oracle reports twelve executions of the unchanged function body under finite principal, authentication, and selector bindings, including two cross-owner private reads. Framework wiring is removed for this oracle, while the function body is retained.

This paired control establishes that a false but permitted MODEL_SEMANTIC premise enabled terminal safety. Prove-



Current-policy fixture control. Both evidence groups are necessary; general semantic or path soundness is unestablished.

Figure 4: Current positive Class-11 control: three fixture-specific non-model premises and four model-semantic premises satisfy the selected vulnerable policy. Both closure and model semantics are necessary to this derivation.

nance and origin-policy admission were preserved; semantic truth was not established. This is not a premise-origin policy violation. The injected error supplies no natural-model semantic error rate, and the fixture-specific non-model closure is not a general closure guarantee.

6.4 A2: Infeasible path admitted as vulnerable

A2 records A2_PATH_COMPATIBILITY_FALSIFIED. In the infeasible arm, a value starts as the constant string "0", is replaced by attacker input only under a fixed Boolean condition c , and is concatenated into an SQL sink only under $\neg c$. The minimally changed feasible control uses c at the sink. The independent oracle enumerates both Boolean values and two attacker values per Boolean: attacker influence at the sink requires $c \wedge \neg c$ in the first arm, whose feasible set is empty, and $c \wedge c$ in the second, whose feasible set is $\{\text{True}\}$. The condition is computed once and unchanged.

The real extraction and production structural-validation route produces and consumes PROVEN_VULNERABLE / STATIC in both arms; both candidates reach RESOLVED_VULNERABLE. The extracted flow/member content is identical. The production deterministic validator supplies all six Class-1 vulnerable requirements. Neither arm uses model evidence or foreign candidate injection. COMPLETE_RELEVANT_PATHS remains UNKNOWN and is not required by this vulnerable policy.

A2 is an executed counterexample to a general feasible-path composition guarantee for the current candidate-local admission route. Conservative candidate enumeration is not itself the falsified claim; promotion of this infeasible composition to a consumed terminal vulnerability is. The result is one intraprocedural Boolean counterexample using mocked request/cursor bindings, not a live database exploit or a population false-positive rate.

6.5 A3: Replacement and revocation capability absent

A3 records A3_UNSUPPORTED_CAPABILITY. Source/caller inspection and executable interface introspection find append-only semantic/memory evidence and coarse universe/fixed-point invalidation, but no supported semantic-fact replacement or retraction route coupled to dependent consumed-proof revocation. The experiment stops at that capability gate.

The current implementation lacks the tested revocation capability. No supported semantic retraction was executed, no mutable-program or fact-replacement test was run, and no stale proof was demonstrated to remain current after such an operation. This is a negative capability finding consistent with the earlier source audit, not a claim that all invalidation mechanisms are absent.

6.6 Historical runtime observation

One separately identified historical Class-2 record, H-R, contains a baseline, provocative probe, matched control, captured outputs, and a consumed runtime-assisted proof. Subject bytes, response/capture hashes, and ledger admission records reconcile. The probe's arithmetic discriminator differs from the literal payload preserved by the matched control, avoiding an interpretation based solely on copying a marker. The selected historical policy derives PROVEN_VULNERABLE / STATIC_AND_RUNTIME from applicability and vulnerable runtime behavior; complete relevant paths remains unknown.

This is a narrow retained runtime-confirmed observation under the recorded executor contract, not a current-identity runtime replication. The executor binary and immutable image contents are unbound, and cleanup is attested rather than independently observed. Another candidate remains planned. No broad runtime completeness or environment-synthesis result follows. Audit groups A14/A15 separate current integration from this historical evidence.

7 Assurance boundaries

7.1 Permitted semantic fallibility

A model can be authorized without being correct. The canonical engine can establish that the producer was allowed to assess a premise, that its citations belong to the packet, and that the resulting status combination satisfies a policy. Those checks do not independently establish sanitizer effectiveness, authorization semantics, or attacker influence. A1 demonstrates the consequence: the admitted false dominance assertion enables a consumed terminal safe decision while its matched truthful assessment remains unresolved (Section 6.3). Correct attribution and permission survive this semantic failure.

Same-premise authoritative precedence is useful but narrower than semantic consistency. It requires applicable coun-

terevidence to exist. It does not detect every inconsistency between different premises or assumptions shared by multiple assessments. The at-least-one-non-model-origin rule likewise prevents purely model-origin terminal support without validating all remaining model semantics. A single trusted premise does not make the entire conjunction true.

The absence of automatically recorded semantic assumptions compounds this interpretive risk. The proof schema permits assumptions, but the canonical path does not populate a complete inventory of model correctness and environmental assumptions. A reader therefore needs the evidence origins and policy context to interpret a finding. This is a limitation of the current implementation, not a recommendation to rename internal enums as a substitute for stronger checking.

7.2 Path-compatible composition

The generic proof engine groups evidence by premise identifier and evaluates requirement sets. A `PremiseAssessment` does not uniformly carry a checked branch condition, path identity, or environment generation per assertion. Packet membership and candidate binding constrain foreign-evidence splicing, but they do not show that selected facts describe one compatible execution.

A2 exercises this actual route and produces a consumed static vulnerable decision for an independently infeasible attacker-to-sink path (Section 6.4). Its feasible control reaches the same terminal. The extractor’s shared flow state and structural validation do not establish joint branch feasibility in this case. The earlier source-based concern is now an executed counterexample, with no model evidence or foreign candidate injection. Class-1 admission does not require the unknown complete-path premise. Candidate identity and permitted evidence origins therefore do not supply a general feasible-path guarantee.

7.3 Dependency freshness and revocation

Changing source, scope, analyzer, registry, or policy inputs can produce a new analysis identity. This isolates incompatible runs, but it is not continuous monitoring. Within an analysis, admitted memory changes invalidate fixed-point completion and schedule conservative review/re-enumeration. The implementation explicitly falls back to global review when no impact relation is available.

A3’s source/caller review and interface inspection establish that the tested semantic replacement/retraction and dependent consumed-proof revocation route is unsupported (Section 6.5). The experiment stops there; it does not execute a supported retraction followed by stale proof reuse. Consumed-proof readers do not universally recheck current semantic or environment generations. Retained Class-11 terminal proofs alongside later unresolved review work demonstrate coarse generation invalidation, without demonstrating that those particular findings became false. A future revocation-capable version must be separately identified and evaluated rather than substituted for this checkpoint.

A1. False permitted semantic premise

False premise: terminal safe.

Matched truthful premise: unresolved.

Admission preserves provenance, not semantic truth.

A2. Infeasible path promoted

$c \wedge \neg c$ and $c \wedge c$: both terminal vulnerable.

Real extraction, validation and admission route.

General feasible-path composition is falsified.

A3. Revocation capability absent

No supported semantic replacement/retraction with dependent consumed-proof revocation.

Capability inspection; no stale-proof execution.

Completed Phase-2A findings for the tested checkpoint.
No population error rates were measured.

Figure 5: Reported completed Phase-2A findings. A1 and A2 execute counterexamples to semantic-truth and general feasible-path interpretations; A3 stops at the unsupported-capability gate.

7.4 Runtime trust

Runtime admission binds the request, candidate, attempt, subject, harness, execution, and captured evidence, with required status and cleanup fields. Such checks are conditional on the executor and environment contract. They cannot independently establish truthful execution, complete reset, or a pinned image merely by validating an attestation’s format and hash.

The current narrow runtime profile and historical observation therefore have a bounded assurance interpretation. Failure to reproduce is a scoped negative observation, not proof of safety. A sound safe conclusion would require an independently justified policy branch and its assumptions. Runtime validation complements admission but does not remove the need to state what the experiment actually discriminates.

8 Durability and residual obligations

The current ledger and workers implement durable candidate/attempt records, leases, stale-attempt fencing, distinct retries, and receipt reconciliation. A terminal outcome is tied to a consumed policy derivation rather than a display grouping or job-success message. Errors, ambiguity, budget exhaustion, and insufficient evidence preserve open work. Missing usage after possible delivery remains unknown; a planned call abandoned before delivery can be known zero. These distinctions prevent a successful process return or a numeric default from standing in for evidence.

Historical diagnostics supply limited operational examples. H-F records a provider-child kill after delivery, a surviving parent, explicit requeue, and a distinct retry for the same candidate. It does not demonstrate death of the parent worker or expired-lease recovery. H-M records valid JSON that violates the semantic response contract; the rejected call supplies no accepted proof/memory linkage. H-X records an issued expansion handle, parent-linked packets, and distinct calls for

one candidate. These cohorts use older analyzer/premise contracts, so they cannot validate the current authority boundary or establish a current durability advantage.

Set-level conservation also has a scope. Recomputing retained candidate identities and reconciling event references is stronger than observing equal row counts, but it cannot establish that every pre-persistence obligation was generated. A complete audit must compare with an independently specified expected population. Persistence is established systems practice; PRISM’s incremental claim concerns attribution and justified discharge under equal persistence and validator capabilities.

9 Evaluation methodology

9.1 Primary hypothesis and comparison

The main hypothesis is that explicit premise-origin and discharge checks reduce unsupported terminal decisions and misattributed residual work while preserving useful validated progress. RQ3 tests authority/admission. RQ8 tests whether relevant dependency changes invalidate consumed decisions, and whether current enforcement meets that requirement. Phase-2A A3 resolves the current capability question negatively: the required replacement/revocation route is unsupported. A dynamic study of a future supported route remains separate.

RQ1 (enumeration), RQ2 (bounded evidence), RQ4 (durability), and RQ5 (cost) are secondary. Enumeration requires an independently labeled discovery population beyond this fixed-candidate design; bounded evidence and cost require quality-matched comparisons. RQ6 (runtime contribution) and RQ7 (smaller-model routing) are deferred until current execution/reproducibility and admission gates are resolved. No completed outcome is assigned to those deferred questions.

The proposed comparison holds fixed the repository, candidates, semantic responses, packets, validators, token/runtime resources, and relevant scheduling/recovery facilities. The baseline is an equally durable queue; the treatment is the current PRISM admission/discharge layer. Recorded-response replay isolates admission from model stochasticity. Any live generation cost is shared provenance, not a claimed replay cost advantage. A separate prospective study would be required to estimate natural model error frequencies and total live costs.

9.2 A credible baseline

The default baseline preserves transactional persistence, durable receipts, retry identities, leases, fencing, source checks, and validator rejection. It uses the same class-specific premise requirement sets as a common decision objective, applies conservative validator precedence, retains conflicts/unknowns, and marks a terminal job only when a complete applicable outcome is supported. It does not add PRISM-specific origin permissions, the non-model-origin floor, or mandatory consumption of a `FindingProof`. An independently validated complete result supplied by a shared validator must be honored in both arms.

The checker/admission interface must be specified before

measurement. If a shared checker already enforces an admission restriction, both arms retain it and the overlap is recorded. If it embodies the entire treatment contract, that component cannot support a separate causal attribution. The baseline must not be forced to trust a checker-rejected response or be deprived of durable state. Conservative alternative reducers provide sensitivity checks; a reducer using the full PRISM admission checker is an equivalence control. Identical behavior under a strong baseline is an informative negative outcome.

9.3 Phase-2A observed falsification results

Table 3 summarizes the completed Phase-2A controls reported in Sections 6.3–6.5. A1 tests an admitted semantic falsehood under matching identities. A2 tests feasible/infeasible source conditions through the actual extraction/validator/admission route. A3 finds the required semantic replacement and revocation capability absent. The prespecified design was sealed before execution according to the supplied reports; Section 5 states the raw-record verification boundary.

These are negative findings about three assurance interpretations of the current implementation. A1 and A2 supply executed counterexamples; A3 supplies a capability-inspection result rather than a stale-proof execution. No fixtures were adjusted to improve outcomes and no PRISM repairs occurred. The findings establish neither population error rates nor comparative benefit. The narrower origin/discharge hypothesis still requires the equally durable, equally equipped matched study below.

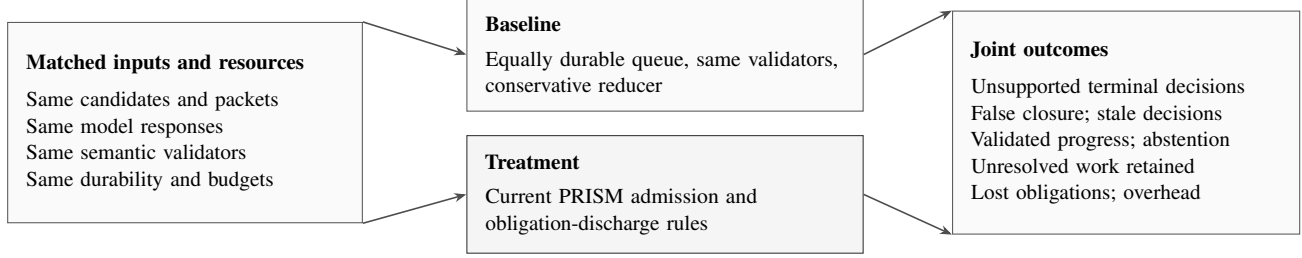
9.4 Outcomes, denominators, and failure rules

The planned full matched protocol additionally injects false model closure, authority-laundering attempts, stale source, policy drift, worker failure, and ambiguous provider delivery. Each injection has a clean matched control and identical inputs across arms. Worker-death tests cover durable planning, submission, receipt, proof creation, terminal commit, expansion, and generation boundaries; provider-child failure remains a distinct condition. Dependency tests separate relevant, irrelevant, and no-change controls and compare affected decisions against fresh recomputation.

Independent adjudication defines program-property truth, path compatibility, affected dependencies, and expected unresolved obligations before arm labels are revealed. PRISM’s own acceptance cannot define the oracle. Unknown labels remain unknown and produce explicit bounds. Repeated attempts are clustered by obligation/instance and repository, not counted as independent vulnerabilities. The fixed-candidate population supports conditional validated recall; undiscovered vulnerabilities require a separate discovery denominator.

The universal-contract claim fails on a verified violating trace within its declared boundary. The comparative-benefit claim requires fewer unsupported decisions or misattributed obligations at acceptable validated progress. An admit-nothing policy cannot succeed merely through apparent precision. Before matched-study execution, preregistration must set practical-effect and recall-noninferiority margins, sample size, clustering, sparse-event uncertainty, and handling of multi-

Proposed comparison; benefit unmeasured



Preserve validator semantics in both arms. More abstention alone does not establish a useful admission benefit.

Figure 6: Proposed matched Phase-2 comparison. Inputs, semantic evidence, validators and resources are held fixed; joint validity, retained progress and accounting outcomes can falsify the incremental-benefit hypothesis.

Table 3: Observed Phase-2A results. Completed experiment reports are incorporated with the verification scope stated in Section 5; no population rates or comparative effect are estimated.

Exp.	Question	Observed result	Interpretation	Limitation
A1	Can a false permitted semantic premise affect a terminal?	False semantic premise enabled terminal safe; truthful control remained unresolved.	Admission preserves provenance, not semantic truth.	Injected semantic error; no natural model-error rate.
A2	Can incompatible paths be promoted to terminal vulnerability?	Yes: infeasible and feasible arms both reached consumed static vulnerable terminals.	General feasible-path composition falsified for this checkpoint.	One intraprocedural Boolean counterexample.
A3	Does supported semantic replacement trigger dependent-proof revocation?	Required replacement/revocation capability absent.	Current dependency-driven semantic revocation claim unsupported.	Capability inspection, not stale-proof execution.

Table 4: Proposed matched-comparison outcomes. Report exact counts and denominators, retain unknown labels, and judge correctness independently of PRISM acceptance.

Outcome	Measurement / denominator
Unsupported vulnerable/safe terminals	Independently unsupported outcomes over respective terminals; also over all scheduled obligations
False closure	Unsupported closure assertions over asserted closures and closure opportunities
Validated progress / recall	Validated resolved obligations and detected labeled instances; fixed candidate population only
Unresolved retained / abstention	Expected unresolved IDs retained; no-terminal obligations over all scheduled obligations
Lost obligations	Independent expected-ID set minus persisted/accounted set at each recovery checkpoint
Stale decisions	Invalidated proofs usable as current over independently affected proofs; historical retention is separate
Overhead / cost	Paired latency, CPU, storage and rework; provider/runtime costs separately, with missing components unknown

ple primary outcomes. If both systems behave identically, shared validators explain the difference, or acceptable progress disappears, the proposed incremental benefit is unsupported. Missing billing makes a cost advantage unidentifiable, not disproven; no cost comparison is completed here.

10 Discussion

The central distinction is operationally consequential: an LLM can be authorized without being correct. Origin restrictions answer who may contribute an assertion; identity checks answer which artifact and attempt it belongs to; class policies answer how assertions authorize a disposition. A semantic checker must still answer whether an assertion follows from the program under the relevant assumptions. Combining these

questions into a single confidence or “proof” label obscures the assurance actually obtained.

Explicit residual uncertainty can improve interpretability even before an accuracy advantage is established. An unresolved obligation with a recorded missing premise has a different meaning from an omitted finding or an interrupted job whose result is unknown. Nevertheless, auditability is not automatic evidence of better security outcomes. It should be measured through recoverable attribution, correct affected-dependency identification, and independent reconstruction, while also measuring overhead and useful progress.

PRISM can be viewed as an integration boundary for stronger static, formal, or runtime evidence. A sound checker could justify specific semantic premises under declared se-

mantics; a scoped runtime discriminator could supply a different kind of evidence. Neither capability should be inferred from the admission schema itself. The strongest defensible current position is a systems investigation into where policy-based admission helps, where existing validators already supply equivalent protection, and where semantic or dependency mechanisms remain necessary.

Phase-2A already supplies negative scientific results. A1 refutes the interpretation of permitted provenance as semantic truth; A2 refutes a general feasible-path guarantee; A3 finds dependency-driven semantic proof revocation unimplemented. Treating these outcomes only as bugs to be repaired would conceal the assurance limits of the experimental subject. A future path-sensitive or revocation-capable PRISM must have a separate system identity. Its proposed repairs cannot replace the observations reported here.

The matched incremental-benefit study can still yield a null or adverse result. If shared validators and durable workflow provide equivalent behavior, the extra layer may primarily make an existing contract explicit. That can aid engineering without establishing incremental research benefit. Conversely, the Phase-2A counterexamples alone do not show that PRISM is ineffective overall. The remaining question concerns measured value of the narrower origin/discharge contract under matched resources, evidence, and useful-progress requirements.

11 Limitations

The tested checkpoint admits a false permitted semantic premise that enables terminal safety and promotes an independently infeasible path to a consumed static vulnerable decision. It lacks the tested semantic-fact replacement/retraction and dependent-proof revocation route. A universal recursive authority argument is also absent, and semantic assumptions are not automatically enumerated in every model-dependent proof. These are observed assurance limits, distinct from proposed future repairs.

A1 uses deterministic injected semantics and a finite fixture oracle; it does not estimate a natural model-error rate. Its fixture-specific closure does not establish a general all-path property. A2 is one intraprocedural Boolean counterexample with mocked request/cursor bindings, not a live database exploit or population false-positive estimate. A3 stops at the unsupported-capability gate and supplies no executed stale-proof failure. The two earlier Class-11 controls use incomplete fixtures and additional validators; they do not establish general authorization correctness.

Independent raw Phase-2A verification remains incomplete because the supplied handoffs omit their linked execution artifacts. Live runtime evidence in this evidence base is historical, with incomplete executor/image identity. Dependency inventories and lower-bounded installation requirements do not seal all transitive packages, native components, or provider behavior.

No completed matched admission study, population recall comparison, cost comparison, competitor head-to-head, or

public enterprise benchmark is available. The registry’s 86 contracts do not imply 86 effective detectors. Earlier numerical reports lacking their exact raw cohort remain excluded. The reports’ three passing preparatory regressions do not establish a recreated full regression suite. Neither incremental benefit nor overall ineffectiveness follows from these bounded controls.

12 Threats to validity

Internal validity. The earlier Class-11 controls and A1 use fixture-specific non-model premises; the earlier negative control withholds deterministic applicability deliberately. A2 instead exercises the production structural validator and extraction route. The Phase-2A reports record a design and fixtures sealed before execution, unchanged checkpoint bytes, no outcome-adjusting changes, and no PRISM fixes. These controls limit outcome tuning but do not remove fixture selection or oracle error. The manuscript audit does not itself rerun the reported experiments or establish a measured success rate.

Construct validity. Policy admissibility, semantic truth, runtime confirmation, and formal proof are different constructs. Raw obligations, vulnerability instances, file/class labels, and attempts are different denominators. A closure predicate is meaningful only relative to its declared scope and trusted abstraction. The matched study must use independent truth rather than equate PRISM acceptance with correctness. A1’s finite oracle retains the function body but removes framework wiring. A2’s Boolean oracle establishes path incompatibility and attacker influence under mocked bindings, not a live SQL exploit. A3 tests supported interface capability, not dynamic proof freshness after retraction.

External validity. Exact authorization fixtures and a narrow historical runtime profile do not represent arbitrary languages, frameworks, dynamic behavior, or deployment configurations. Fixing the candidate population deliberately excludes discovery effects. Supported-scope and setup failures must remain visible in any later extension.

Reproducibility. Source/package identities were verified, but the full execution environment was not recreated. The source distribution omits some support files present in the complete checkpoint, dependencies are not transitively hash-locked, and runtime images/executables are incompletely bound. Artifact consistency is distinct from behavioral reproducibility. The Phase-2A handoffs’ shared checkpoint and policy identifiers were checked, but their linked seal, raw results, oracle traces, pre/post inventories, proof objects, and interface inventory remain unavailable for independent verification. Claims about execution controls and replay are attributed to those reports, not to a replay performed for this revision.

Model/provider validity. Current receipts identify a requested model alias and CLI version rather than an exposed immutable backend snapshot. Service-side retries, billing, and ambient execution context are incompletely observable. Packet bounds do not establish total context, generated-output or billing bounds; the requested CLI output cap was not en-

forced. Recorded-response replay can isolate admission behavior but cannot estimate future live-model reliability or comparative inference cost. Phase-2A uses no live provider calls; A1’s injected semantic falsehood therefore says nothing about how often a real model makes that error.

Benchmark contamination. Public vulnerability tasks may have appeared in training data, and disclosed bug descriptions can change the task from discovery to reproduction. Later benchmarks must document exposure, task framing, revision identity, and grading uncertainty rather than transfer published scores directly to PRISM.

Fixture bias. Hand-selected short controls favor inspectable local shapes and exclude many realistic interactions. Fixture-specific validators limit generalization for the earlier Class-11 controls and A1; this caveat must not obscure that A2’s negative result uses the production structural validator. The sealed selection and minimal feasible control support a falsification test, without supporting a representative population denominator.

Historical evidence compatibility. Historical worker/runtime diagnostics differ in analyzer, registry, policy, or execution identity. Shared subject bytes do not make cohorts interchangeable. Their role is to document narrowly observed behavior and motivate current controls, not to enlarge a current sample or support a current comparative claim.

13 Conclusion

PRISM implements a candidate-local contract linking identity-bound obligations, semantic questions, premise origins, evidence, class policies, and consumed terminal derivations. The audit supports this implementation account. Completed Phase-2A controls show that a false permitted semantic premise enabled terminal safety, an independently infeasible attacker-to-sink path received a consumed static vulnerable decision, and the tested semantic replacement/dependent-proof revocation capability is absent. Provenance is not semantic truth; current generic admission does not guarantee feasible-path composition; dependency-driven semantic proof revocation is not implemented.

These negative results characterize the tested checkpoint and cannot be replaced by future repairs. They do not estimate population rates, overall ineffectiveness, or comparative benefit. Independent verification of the linked raw Phase-2A records remains outstanding. The unresolved research hypothesis is whether the narrower origin/discharge contract provides incremental value over an equally durable, equally equipped baseline while preserving useful progress. That claim requires the matched study and a reproducible artifact boundary.

A Remaining Research Gates

1. Obtain and independently verify the linked raw Phase-2A result records, execution seal, oracles, pre/post inventories, proof objects, ledgers, and interface inventory. The completed reported A1/A2/A3 outcomes are already incorporated; checkpoint identity compatibility passes.
2. Execute the preregistered matched admission comparison against an equally durable baseline with the same evidence, validators, and resources; retain null and adverse outcomes.
3. Retain a current reproducible regression transcript with source, dependency, runtime, and command identities.
4. Preserve every observed failure and its original evidence before any repair.
5. Treat semantic, path, or revocation repairs as separately versioned PRISM revisions and separate comparison arms.
6. Complete publication sanitization and disclosure approval before external distribution or submission.
7. Finalize the external benchmark strategy, separating discovery, reproduction, admission validity, and runtime assurance with independently defined denominators.

B Premise Authority Catalog

The audited catalog contains 37 premise types. Model-assessable means that `MODEL_SEMANTIC` may supply an assessment only when the question is issued and the selected class policy permits it. It does not grant authority for every class or certify semantic correctness.

The ordinary non-model origins are `PARSER`, `STATIC_ANALYZER`, `DETERMINISTIC_RULE`, `RUNTIME`, `ENVIRONMENT`, and `HUMAN_REVIEW`, each subject to class-policy restrictions. Runtime-only and environment-only entries override ordinary permissions. `SYSTEM_STATIC` is conceptual shorthand, not a stored origin enum. The catalog is a permission inventory, not a detector inventory.

C Experiment Identities and Cohort Boundaries

Table 1 records complete current checkpoint, analyzer, registry, policy, and analysis digests. The analyzer digest was reconstructed using recorded runtime metadata, without recreating that runtime. The positive and negative Class-11 controls were recorded on 28 August 2026. They share the inspected analyzer, registry, and authorization policy, but differ in subject, analysis, and model-policy identities. They are not repeated executions of one immutable analysis.

For each control, verification reconciled the candidate identity, packet and control bindings, accepted response digest, initial and final proof records, consumed-proof pointer, and attempt lineage. Each ledger retained one terminal focal candidate, three planned candidates, and three unresolved review obligations. Later review generations remain unfinished; the original consumed proof is retained. Neither the recorded terminal state nor the identity match establishes whole-repository completion or continuing dependency freshness.

Historical diagnostics form separate cohorts: H-F records

provider-child failure with a surviving parent; H-M records malformed semantic-contract rejection; H-X records one bounded evidence expansion. Their older semantic question policies do not verify the current model-forbidden-premise boundary. H-R is the separately retained historical Class-2 runtime observation, with an incompletely bound executor/image and attested cleanup. Shared subject bytes would not make these historical cohorts compatible with a current analyzer or policy. Their observations are not pooled with the two current controls.

Phase-2A adds four reported scan arms and one capability check on the same checkpoint, separately from the retained and historical cohorts. A1 records one consumed safe terminal; A2 records two consumed vulnerable terminals; A3 stops at the unsupported-capability gate. No full matched-comparison result or population performance denominator is included.

C.1 Phase-2A exact result identities

The reports agree on classifications. The exact result IDs below are from the supplied paper-facts report; their linked JSON preimages were unavailable for independent recomputation.

A1: `A1_POLICY_ADMISSION_ACCEPTS_FALSE_SEMANTIC`
cdd5ac440770ec516420216b30b398e034040bc308ee6d60b8f577cf6ee99aa4

A2: `A2_PATH_COMPATIBILITY_FALSIFIED`
c4fa406f8a27186f9c2e3d176c62c90c04ef333fd9751ba9047a0d40310f8539

A3: `A3_UNSUPPORTED_CAPABILITY`
58f2a3a771078d6aaf85fbefa0527249f78ffbc46206dbeeda50e28f5d491106

The reported design digest is 289f017d44a93cef184b583c9377a97e06c074099fb16f19f1850acf146a5ec0. Its sealed preimage was unavailable. The shared source-tree digest, independently reconstructed from the retained checkpoint, is 35c2f2fa38faae79e2f907591b3ca6768ee41acc985358fecb16e7ab9f86aaf0. Table 1 gives the other shared identities.

A1's source matches the retained Class-11 fixture, with digest bf0b45b4f6bdf5d2cdb03e7e72a273375b0835979145138f027d3471d36611f4. The independently reconstructed Class-1 policy digest for A2 is afbbffcc9325743f415162adb99a25ed824b78351b8fcf4a2fd9e9e2127918e3; the shared Class-11 policy is in Table 1.

C.2 Supplied report identity and replay boundary

The received paper-facts file has 9,608 bytes and SHA-256 ba9e99b30d4272a33209c058bfe3d05e0d08357b3a06b2931caacf0a40e5817e. The summary has 4,572 bytes and SHA-256 a84dd66658b18ef01d32bdfefee97f1b064a04c53d206a53a9661ef78f2162580. These hashes identify the supplied handoffs, not their missing linked records. The reports state that six proof objects replayed, four SQLite integrity checks and three terminal consumption checks passed, and three preparatory regressions passed. These are reported checks, not new replay or regression results from the manuscript revision. Raw-record verification and environment reconstruction remain open.

Table 5: Complete premise catalog at the audited checkpoint. O: ordinary non-model origins subject to class policy; R: runtime only; E: environment only. A model-assessable entry remains subject to the issued question and class policy.

Premise	Model-assessable	Non-model category
CANDIDATE_APPLICABLE	Yes	O
REPOSITORY_SCOPE_COMPLETE	No	O
ATTACKER_CONTROLLED_INPUT	Yes	O
REACHABLE_FLOW	Yes	O
COMPLETE_RELEVANT_PATHS	No	O
SECURITY_SENSITIVE_OPERATION	Yes	O
SECURITY_RELEVANT_INFLUENCE	Yes	O
EFFECTIVE_MITIGATION_ABSENT	Yes	O
EFFECTIVE_MITIGATION_PRESENT	Yes	O
OPERATION_UNREACHABLE	No	O
NON_SECURITY_SEMANTICS	Yes	O
AUTHENTICATED_PRINCIPAL	Yes	O
ATTACKER_RESOURCE_SELECTOR	Yes	O
SENSITIVE_ACTION	Yes	O
AUTHORIZATION_PREDICATE_ABSENT	Yes	O
AUTHORIZATION_PREDICATE_DOMINATES	Yes	O
SECURITY_RESPONSIBILITY_LOCAL	Yes	O
INSECURE_CONFIGURATION	Yes	O
CONTROL_SCOPE_COMPLETE	No	O
REQUIRED_CONTROL_ABSENT	Yes	O
REQUIRED_CONTROL_PRESENT	Yes	O
SENSITIVE_MATERIAL	Yes	O
MATERIAL_EXPOSED	Yes	O
DEPENDENCY_PRESENT	Yes	O
ADVISORY_APPLICABLE	Yes	O
ADVISORY_NOT_APPLICABLE	No	O
BUSINESS_INVARIANT_DEFINED	Yes	O
BUSINESS_INVARIANT_VIOLATED	Yes	O
STATE_TRANSITION_REACHABLE	Yes	O
DANGEROUS_CAPABILITY	Yes	O
TRUST_BOUNDARY_CROSSED	Yes	O
OBSERVATION_CONFIRMED	Yes	O
RUNTIME_BEHAVIOR_VULNERABLE	No	R
RUNTIME_BEHAVIOR_DEFENDED	No	R
ENVIRONMENT_CONTROL_KNOWN	No	E
ENVIRONMENT_BEHAVIOR_VULNERABLE	No	E
ENVIRONMENT_BEHAVIOR_DEFENDED	No	E

References

- [1] Anthropic. Making frontier cybersecurity capabilities available to defenders. Vendor technical announcement, February 2026. URL <https://www.anthropic.com/news/claude-code-security>. February 20, 2026; Claude Code Security launch description; accessed September 8, 2026.
- [2] Dirk Beyer and Marie-Christine Jakobs. CoVeriTest: Cooperative verifier-based testing. In *Fundamental Approaches to Software Engineering*, volume 11424 of *Lecture Notes in Computer Science*. Springer, 2019. doi: 10.1007/978-3-030-16722-6_23. URL https://link.springer.com/chapter/10.1007/978-3-030-16722-6_23. FASE 2019.
- [3] Dirk Beyer, Thomas A. Henzinger, M. Erkan Kermoglu, and Philipp Wendler. Conditional model checking: A technique to pass information between verifiers. In *Proceedings of the ACM SIGSOFT 20th International Symposium on the Foundations of Software Engineering, FSE '12*. Association for Computing Machinery, 2012. doi: 10.1145/2393596.2393664. URL https://www.sosy-lab.org/research/pub/2012-FSE.Conditional_Model_Checking.pdf. Article 57.
- [4] Capital One Tech. Announcing VulnHunter. Vendor technical announcement, July 2026. URL <https://www.capitalone.com/tech/open-source/announcing-vulnhunter/>. July 16, 2026; open-source system with vendor-reported capabilities; accessed September 8, 2026.
- [5] Cognition. Agentic MapReduce. Vendor technical article, July 2026. URL <https://devin.ai/blog/agentic-map-reduce>. July 1, 2026; accessed September 8, 2026; vendor-reported system.
- [6] GitHub. About data flow analysis. CodeQL official documentation. URL <https://codeql.github.com/docs/writing-codeql-queries/about-data-flow-analysis/>. Undated, mutable documentation; accessed September 8, 2026.
- [7] Jinyao Guo, Chengpeng Wang, Xiangzhe Xu, Zian Su, and Xiangyu Zhang. RepoAudit: An autonomous LLM-agent for repository-level code auditing. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 21083–21100. PMLR, 2025. URL <https://proceedings.mlr.press/v267/guo25n.html>.
- [8] Nahum Korda and Gadi Evron. OpenAnt: LLM-powered vulnerability discovery through code decomposition, adversarial verification, and dynamic testing, 2026. URL <https://arxiv.org/abs/2606.19149v2>. Preprint; version 2, June 18, 2026.
- [9] Hwiwon Lee, Ziqi Zhang, Hanxiao Lu, and Lingming Zhang. SEC-bench: Automated benchmarking of LLM agents on real-world software security tasks. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=QQhQIqons0>. Author manuscript arXiv:2506.11791v2, October 22, 2025.
- [10] Hwiwon Lee, Jiawei Liu, Dongjun Kim, Wubing Xia, Ziqi Zhang, Chunqiu Steven Xia, and Lingming Zhang. SEC-bench Pro: Can language models solve long-horizon software security tasks?, 2026. URL <https://arxiv.org/abs/2605.26548v2>. Preprint, version 2, July 20, 2026; first submitted May 26, 2026.
- [11] Haonan Li, Yu Hao, Yizhuo Zhai, and Zhiyun Qian. Enhancing static analysis for practical bug detection: An LLM-integrated approach. *Proceedings of the ACM on Programming Languages*, 8(OOPSLA1), 2024. doi: 10.1145/3649828. URL https://www.cs.ucr.edu/~zhiyunq/pub/oopsla24_llift.pdf. Article 111.
- [12] Ziyang Li, Saikat Dutta, and Mayur Naik. IRIS: LLM-assisted static analysis for detecting security vulnerabilities. In *International Conference on Learning Representations*, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/582d4e27fa24168f3af1f4582655034b-Abstract-Conference.html.
- [13] Jacqueline L. Mitchell and Chao Wang. Agentic Interpretation: Lattice-structured evidence for LLM-based program analysis, 2026. URL <https://arxiv.org/abs/2605.12694v1>. Preprint, version 1, May 12, 2026.
- [14] George C. Necula. Proof-carrying code. In *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL '97*, pages 106–119. Association for Computing Machinery, 1997. doi: 10.1145/263699.263712. URL <https://doi.org/10.1145/263699.263712>.
- [15] OpenAI. Codex Security CLI FAQ. Official technical documentation. URL <https://learn.chatgpt.com/docs/security/cli/faq>. Undated, mutable vendor documentation; accessed September 8, 2026.
- [16] Visa. Visa Vulnerability Agentic Harness: Architecture. Public repository technical documentation. URL <https://github.com/visa/visa-vulnerability-agentic-harness/blob/main/docs/architecture.md>. Undated document on mutable main branch; no immutable commit pinned; accessed September 8, 2026.
- [17] Chengpeng Wang, Wuqi Zhang, Zian Su, Xiangzhe Xu, Xiaoheng Xie, and Xiangyu Zhang. LLMDFA: Analyzing dataflow in code with large language models. In *Advances in Neural Information Processing Systems*, volume 37, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/ed9dcde1e

- [18] Chengpeng Wang, Wuqi Zhang, Zian Su, Xiangzhe Xu, and Xiangyu Zhang. Sanitizing large language models in bug detection with data-flow. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 3790–3805. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.findings-emnlp.217. URL <https://aclanthology.org/2024.findings-emnlp.217/>.
- [19] Chengpeng Wang, Yifei Gao, Wuqi Zhang, Xuwei Liu, Jinyao Guo, Mingwei Zheng, Qingkai Shi, and Xiangyu Zhang. NESA: Relational neuro-symbolic static program analysis. *Proceedings of the ACM on Software Engineering*, 3(FSE), 2026. doi: 10.1145/3808161. URL <https://doi.org/10.1145/3808161>. Article FSE154. Author manuscript: arXiv:2412.14399v2.
- [20] Haoze Wu, Clark Barrett, and Nina Narodytska. LEMUR: Integrating large language models in automated program verification. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=Q3YaCghZNt>. Author preprint: arXiv:2310.04870v5.
- [21] Fabian Yamaguchi, Nico Golde, Daniel Arp, and Konrad Rieck. Modeling and discovering vulnerabilities with code property graphs. In *2014 IEEE Symposium on Security and Privacy*. IEEE, 2014. doi: 10.1109/SP.2014.44. URL <https://prosec.mlsec.org/docs/2014-ieeeesp.pdf>.